05 — Algorithm Patterns for Q4

Q4 needs one *idea*. The good news: on the GCA it's almost always one of the five patterns below. (And remember: Q4 is your **bonus** question — 3/4 already passes. Don't panic here.)

---

0. When is brute force OK?

"Brute force" = the obvious, simple way (often a double loop), even if it's slow. **Whether it's fast enough depends on the input size n** the problem promises. Rough rule for Python (it does very roughly 10–100 million simple operations per second):

Input size n	Safe approach	Meaning
n ≤ 1,000	$O(n^2)$ — a double loop is fine	**just brute force it!**
n ≤ 100,000	O(n log n) or O(n)	sort once, or a single pass with dict/set
n ≤ 10,000,000	O(n) only	a single pass, no nesting

> **"$O(n^2)$" just means "a loop inside a loop"** — for n items you do about n × n steps. If the problem says `n ≤ 1000`, that's a million steps — instant. So **check the limits first**: if they're small, don't be clever, be correct. A slow-but-right answer scores; a clever-but-broken one scores zero.

---

1. Dictionary / Set lookups (the most common Q4 — see file 03)

"Have I seen X before?", counting, pairs summing to a target. This is pattern #1 for a reason — reach for it first. Full explanations and code are in file 03.

---

2. Prefix sums — add up any range instantly

The problem it solves: "what's the sum of the items between position i and j?" — asked many times, or over many different ranges. Re-adding each range is slow. A **prefix-sum array** stores running totals so any range sum becomes one subtraction.

```
nums = [3, 1, 4, 1, 5]
prefix = [0]                              # prefix[k] = sum of the FIRS
for x in nums:
    prefix.append(prefix[-1] + x)     # each entry = previous total
# prefix is now [0, 3, 4, 8, 9, 14]
# sum of nums[i..j] inclusive  ==  prefix[j+1] - prefix[i]
```

Why it works: `prefix[j+1]` is "everything up to j" and `prefix[i]` is "everything before i." Subtract, and only the middle chunk survives. Sum of `nums[1..3]` = `prefix[4] - prefix[1]` = 9 - 3 = 6 (which is 1+4+1 ✓). Each range answer is now ONE subtraction instead of a loop.

Count subarrays summing to k (classic Q4 — prefix sums + a dict together):

```
def subarray_sum_count(nums, k):
    count, running, seen = 0, 0, {0: 1}   # seen = {running total :
    for x in nums:
        running += x                       # running total up to he
        count += seen.get(running - k, 0)  # how many earlier totals
        seen[running] = seen.get(running, 0) + 1
    return count
```

Idea: if two running totals differ by exactly k, the chunk between them sums to k. So we remember every running total we've seen and ask "have I seen one that's k smaller?"

---

3. Two pointers — sorted data, or scanning from both ends

Idea: instead of one moving index, use **two** — often one at each end — and move them toward each other based on what you find.

```
# Is there a pair that sums to target, in a SORTED list?
def has_pair(nums, target):
    lo, hi = 0, len(nums) - 1        # one finger at the start, one
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target: return True
        if s < target:  lo += 1      # sum too small → grow it → mo
        else:           hi -= 1      # sum too big   → shrink it → 
    return False
```

Why it needs a sorted list: because it's sorted, moving `lo` right *always* increases the sum and moving `hi` left *always* decreases it — so each step is a confident decision. This turns a double loop ($O(n^2)$) into a single sweep ($O(n)$). Also used for: reversing in place, merging two sorted lists, "container"-style problems.

---

4. Sliding window — best/longest of a CONTIGUOUS chunk

Idea: keep a "window" (a start and end) over a contiguous stretch. **Grow** it on the right; when it breaks a rule, **shrink** it from the left. Two fingers again, both moving rightward, never backward — so it's fast.

```
def longest_unique_substring(s):
    seen = set()                      # characters currently inside
    left = best = 0
    for right in range(len(s)):       # right edge sweeps across the
        while s[right] in seen:       # new char already inside? wi
            seen.remove(s[left])      # shrink from the left until
            left += 1
        seen.add(s[right])            # now safe to include the new
        best = max(best, right - left + 1)   # window length = right
    return best
```

Trace `"abca"`: window grows `a`, `ab`, `abc` (best 3); at the second `a`, shrink off the left `a`, then include the new one → window `bca` (still 3). Answer 3. Use this whenever you see **"longest / shortest contiguous substring or subarray with property X."**

Fixed-size window (size k): slide it along — add the element entering on the right, remove the one leaving on the left, update your best. No inner `while` needed.

---

5. Sort first, then it's easy

Many problems collapse once the data is in order:

- **min difference** between any two values → sort, then only check neighbors
- **"can we pair everyone up"** → sort both lists, compare position by position
- **k largest** → `sorted(nums, reverse=True)[:k]`
- **custom order** → `sorted(items, key=lambda x: (x[1], -x[0]))` (read: "sort by the 2nd field; break ties by the 1st field, descending")

Sorting costs O(n log n) — cheap — and often makes the rest trivial. If you're stuck, ask: "*would this be easier if the data were sorted?*"

---

6. Greedy — grab the best local choice each step

Idea: build the answer by always taking the choice that looks best *right now* (biggest, smallest, closest), without looking ahead. E.g. "minimum number of coins/moves/steps" often means: sort, then repeatedly take the largest that fits.

> **Warning:** greedy is fast but not always correct — the best local move isn't always the best overall. **Test a greedy idea against the given examples before trusting it.** If it fails an example, fall back to another pattern.

Recognition table — phrase → pattern

The problem says...	Try
"seen before / duplicate / count / frequency"	dict or set
"sum of a range / subarray sum"	prefix sums
"sorted array + pair / target"	two pointers
"longest / shortest contiguous substring or subarray"	sliding window
"minimum difference / closest pair / matching"	sort first
"n ≤ 1000"	just brute force

Exercises (in `practice/q4_problem_solving.py`, second half)

1. `max_subarray_sum_k(nums, k)` — max sum of any window of size k (fixed sliding window).
2. `smallest_difference(a, b)` — closest pair between two lists (sort + two pointers).
3. `count_pairs_with_sum(nums, target)` — count pairs adding to target (dict).
4. `equal_split_index(nums)` — index where left sum == right sum (prefix sums).

Which idea for each? 1 → sum the first k, then slide (add new, subtract old), tracking the max. 2 → sort both, walk two pointers, keep the smallest gap. 3 → the Two Sum "seen partner" idea with a count. 4 → total sum, then sweep keeping a left-running sum; the split is where `left == total - left - nums[i]`.